Parallelizing DEVS Traffic Model for Distributed Systems

Author–Hassan Haghighia,, Maamar El Amine Hamria,, Thi Phuong Kieua

10460 : Priyan Almeida

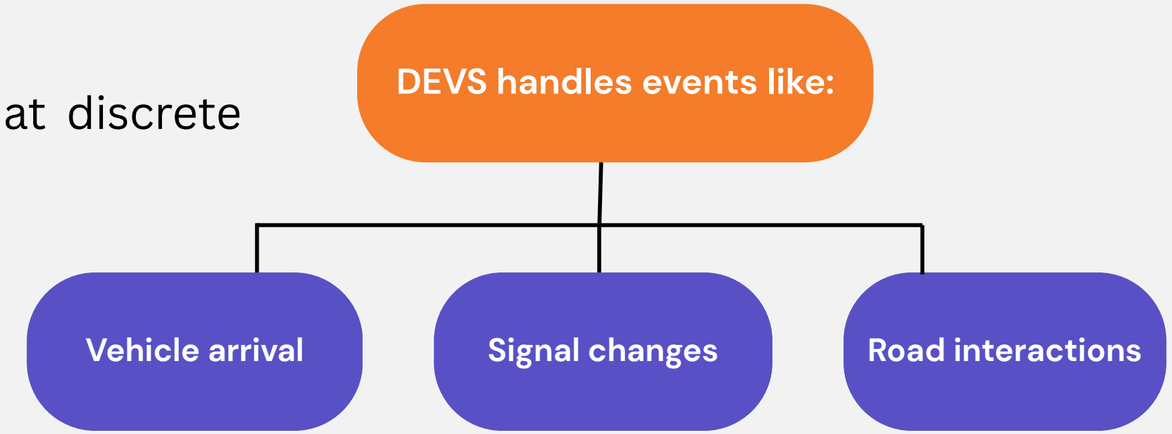
10461 : Sherwin Dmonte

10465 : Snowil Tuscano

Prof. Ashok Kanthe

Introduction

- Discrete Event System Specification is a mathematical modeling framework
- Used to model systems where state changes occur at discrete points in time
- Traffic systems are complex and event-driven
- Simulation helps analyze traffic flow and congestion



Problem Statement

Traditional traffic simulation using Discrete Event System Specification is computationally expensive and inefficient for large-scale systems due to its sequential execution nature.

Difficulties that arise:

- High execution time for large traffic networks
- Lack of scalability with increasing number of vehicles/events
- Inability to support real-time traffic simulation efficiently

Objective

- Improve performance of traffic simulation models
- Reduce total execution time of simulations
- Enable handling of large-scale traffic systems
- Apply computing techniques
  - Parallel Computing
  - Distributed Computing
- Maintain accuracy and correct event sequencing
- Optimize CPU and resource utilization
- Support real-time and near real-time simulations

DEVS Traffic Components

Traffic system is divided into independent modules for simulation

AMGenerator

Generates vehicles at defined intervals

Controls traffic input rate

Simulates entry of vehicles into the system

AMRoad

AMRoad represents road segment behavior

Handles:

- Vehicle movement
- Queue formation
- Delay calculation

Core processing unit of simulation

AMTransducer

AMTransducer collects simulation results

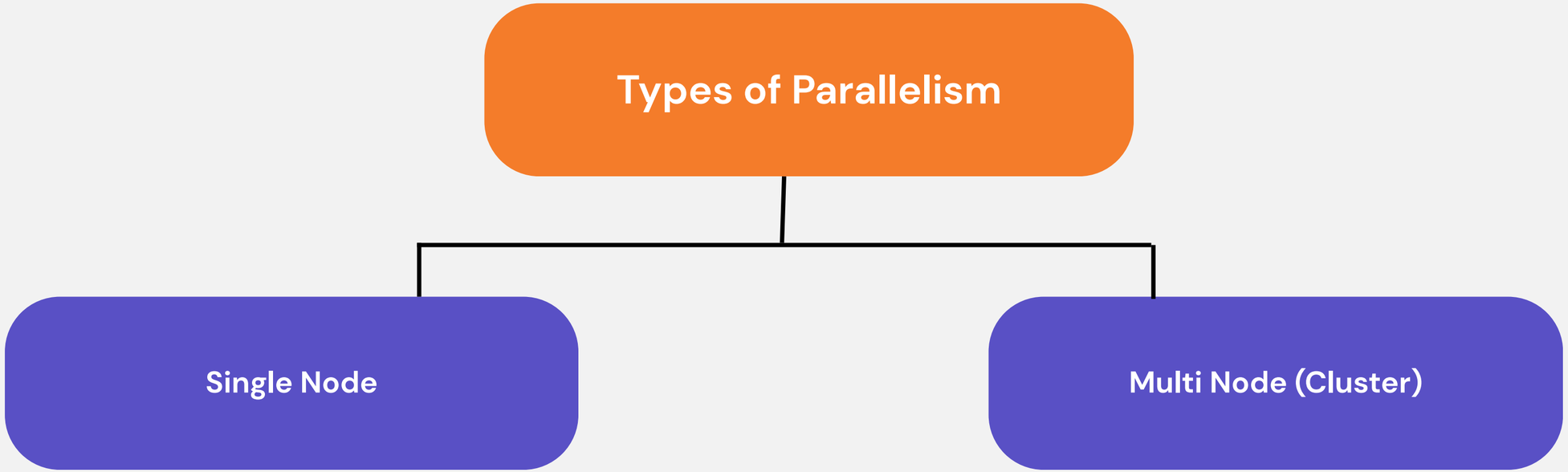
Measures:

Throughput
Waiting time
Performance metrics

Why Parallelization?

In traditional traffic simulation, everything runs one step at a time (sequential execution).
This makes the process slow, especially when the system becomes large and complex.
When we simulate real-world traffic with thousands of vehicles,
the system needs to handle a huge amount of calculations, which increases processing time.
To solve this, we use parallelization, where multiple tasks are executed at the same time.
This approach helps to:

- Reduce execution time by running tasks simultaneously
- Use system resources efficiently (CPU cores, machines)
- Handle large-scale traffic systems easily (scalability)



Single Node Parallelism

- Parallel execution within one machine (single system)
- Uses multiple CPU cores to run tasks at the same time

Implementation

- Uses Java ExecutorService for multithreading
- Creates multiple threads inside a single JVM
- Threads share:
  - CPU
  - Memory
- No need for network or distributed setup

Parallel Components

  - AMGeneratorGenerates vehicles
  - AMRoadHandles road logic (queue, delay)
  - AMTransducerCollects simulation results

All these components run simultaneously in different threads

Single-node parallelism improves speed by running multiple tasks at the same time within one system, but scalability is limited.

Multi-Node (Cluster) Parallelism

- Parallel execution across multiple machines (nodes)
- Each node handles part of the simulation

Working

- Simulation components distributed across nodes:
  - Generator
  - Road segments
  - Transducer
- Nodes work independently
- Coordinator ensures:
  - Synchronization
  - Correct event order

Communication

- Nodes communicate using:
  - Message passing
  - Network protocols

Multi-node parallelism distributes work across multiple machines, making the system highly scalable and powerful for large traffic simulations.

Technologies Used

Apache Kafka

- Used for data communication between components
- Supports real-time data streaming
- Helps in connecting distributed nodes efficiently

Akka

- Uses actor-based model
- Each component works like an independent unit
- Enables asynchronous communication

MPI (Message Passing Interface)

- Used for high-performance computing
- Allows processes to communicate across machines
- Suitable for parallel simulations

Hadoop / Spark

- Used for large-scale data processing
- Handles big traffic datasets efficiently
- Supports distributed computing

Discrete Event System

A deterministic finite automaton is defined as a 5-tuple:
$G = (Q, \Sigma, \delta, q0, Qm)$ (2)
where:

- Q: finite set of states
- Σ: finite set of events
- $\delta : Q \times \Sigma \to Q$: partial transition function
- $q0 \in Q$: initial state
- $Qm \subseteq Q$: set of marked. (accepting) states

Given two DFAs: $G1 = (Q1, \Sigma1, \delta1, q01, Qm1)$, and $G2 = (Q2, \Sigma2, \delta2, q02, Qm2)$, their synchronous product

$$\mathbf{G1 \mathbin{/\!/} G2 = (Q1 \times Q2, \Sigma1 \cup \Sigma2, \delta12, (q01, q02), Qm1 \times Qm2)\ (6)}$$

Single Node Parallelism Example

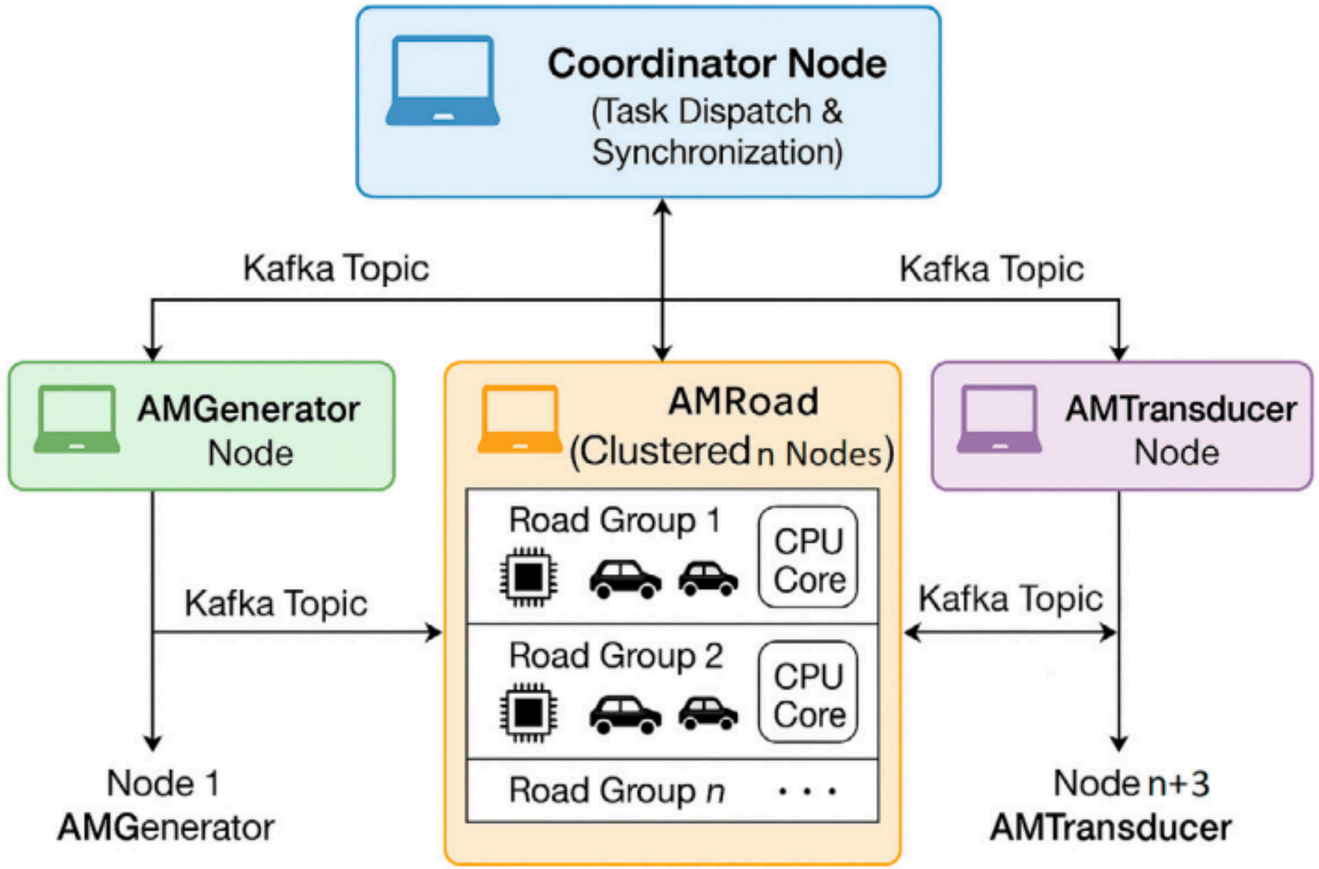


Fig. 1: Distributed architecture for DEVS-Based traffic simulation

Comparison of Single Node and Multi-Node Parallelism

Specification	Single Node	Multi-Node (Cluster)
Number of nodes	1 machine (local)	Multiple machines (distributed)
Concurrency model	Multithreading	Distributed tasks with coordination
Scalability	Limited by local resources	High scalability, horizontal scaling
Communication	In-memory/shared memory	Network-based (e.g., MPI, Kafka)
Performance	Constrained by hardware	Potential real-time performance
Use cases	Lightweight simulations	Large-scale, data-intensive simulations

Future Scope

- Integration with real-time traffic data from sensors and IoT devices
- Development of smart city traffic management systems
- Use of AI/ML for:
  - Traffic prediction
  - Congestion control
- Implementation on large-scale cloud platforms for better scalability
- Improvement in synchronization techniques for distributed systems
- Extension to multi-lane and complex road networks
- Support for autonomous and connected vehicle systems
- Optimization of communication overhead in cluster environments

Conclusion

- Parallelizing DEVS model improves overall performance
- Helps in reducing simulation execution time significantly
- Enables handling of large-scale traffic systems efficiently
- Supports both:
  - Single-node (multithreading)
  - Multi-node (distributed systems)
- Cluster-based architecture provides:
  - Better scalability
  - Faster processing
  - Efficient resource utilization

Parallel DEVS is a powerful approach for building fast, scalable, and efficient traffic simulation systems.

Thank You !